Raíces del Fogón — Documentación técnica

Sistema de información web para restaurante de comida tradicional colombiana

Institución	Colegio Parroquial San Luis Gonzaga
Programa	Técnico en Programación de Software — SENA, Regional Distrito Capital, Centro de Servicios Financieros
Ficha	3166425
Integrante	Valentina Combita Durango
Versión del documento	1.1 — octubre de 2026 (agrega la subida de fotos de platos)

> **Nota sobre la identidad corporativa.** La misión, la visión, los objetivos y los valores que aparecen en este documento y en la página **Nosotros** son una **propuesta pendiente de validación por el equipo**. Se redactaron a partir del concepto del proyecto y se pueden ajustar en `src/empresa.js`. Los datos de contacto (dirección, teléfono, correo y horario) **no se incluyeron porque no estaban disponibles**; la sección de contacto aparece en la página en cuanto se completan en ese mismo archivo.

1. Descripción del proyecto

Raíces del Fogón es un restaurante de comida tradicional colombiana. Su sistema de información web permite:

- A los **visitantes**: consultar el menú, buscar platos y filtrarlos por categoría, y conocer la empresa (misión, visión, objetivos, valores y servicios).
- A los **clientes registrados**: armar una orden, enviarla como pedido, consultar su historial y cancelar los pedidos que sigan pendientes.
- Al **administrador**: gestionar platos y categorías (crear, consultar, editar y eliminar), subir la foto de cada plato desde su equipo, gestionar los pedidos y su estado, y consultar reportes de ventas.

2. Problema o necesidad

El restaurante necesitaba dar a conocer su menú y recibir pedidos sin depender de llamadas o de menús impresos que se desactualizan. La versión anterior del proyecto tenía un menú fijo escrito en el código (`src/platos.js`), sin base de datos, sin registro de usuarios y sin forma de guardar los pedidos. La evaluación de la muestra empresarial del SENA (24-09-2026) señaló como pendientes la identidad corporativa, la base de datos y sus relaciones, los formularios de registro, el CRUD, las consultas y el manual de usuario.

3. Objetivos *(propuesta pendiente de validación)*

Objetivo general. Desarrollar un sistema de información web que permita a Raíces del Fogón dar a conocer su menú, gestionar sus platos y categorías, y registrar y hacer seguimiento a los pedidos de sus clientes.

Objetivos específicos.

1. Publicar el menú del restaurante con búsqueda y filtros por categoría.
2. Permitir el registro y el inicio de sesión seguro de los clientes.
3. Registrar los pedidos y su detalle en una base de datos relacional MySQL.
4. Brindar al administrador la gestión completa (crear, consultar, editar y eliminar) de platos y categorías.
5. Permitir el seguimiento del estado de los pedidos por parte del cliente y del administrador.
6. Generar consultas y reportes de ventas que apoyen la toma de decisiones.

4. Identidad corporativa *(propuesta pendiente de validación)*

Misión. Ofrecer platos tradicionales de la cocina colombiana, preparados con recetas de nuestras regiones e ingredientes frescos, en un ambiente cálido y familiar, y facilitar que nuestros clientes conozcan el menú y hagan sus pedidos de forma sencilla y segura a través de nuestra plataforma web.

Visión. Para el año 2030, ser reconocidos en Bogotá como un restaurante referente de la cocina tradicional colombiana, destacado por la calidad de sus platos, la atención cercana a sus clientes y el uso de la tecnología para agilizar sus pedidos.

Valores. Tradición, Calidad, Hospitalidad, Honestidad, Responsabilidad y Trabajo en equipo (la descripción de cada uno está en la página **Nosotros**).

Servicios. Menú en línea, pedidos por la página y seguimiento del pedido.

5. Requisitos

5.1 Requisitos funcionales

Código	Requisito	Dónde se cumple
RF01	Mostrar el menú con foto, descripción, categoría y precio	Página Menú (`/`)
RF02	Filtrar el menú por categoría y buscar por texto	Página Menú
RF03	Registrar clientes con nombre, correo, teléfono y contraseña	`/registro`
RF04	Iniciar y cerrar sesión	`/login` , botón **Cerrar sesión**
RF05	Armar una orden (agregar, sumar, restar y quitar platos, y vaciarla)	Panel **Orden** del menú
RF06	Enviar la orden como pedido (solo con sesión iniciada)	Botón **Pedir**
RF07	Consultar el historial y el detalle de los pedidos propios	`/mis-pedidos`
RF08	Cancelar un pedido propio mientras esté pendiente	Detalle del pedido
RF09	CRUD de platos con búsqueda, filtros combinados y orden	`/admin/platos`
RF10	CRUD de categorías	`/admin/categorias`
RF11	Consultar todos los pedidos con filtros por cliente, estado y fechas	`/admin/pedidos`
RF12	Cambiar el estado de un pedido	Detalle del pedido (administrador)
RF13	Ver reportes: pedidos, ventas, ticket promedio, platos más vendidos	`/admin/reportes`
RF14	Mostrar misión, visión, objetivos, valores y servicios	`/nosotros`
RF15	Subir, reemplazar y quitar la foto de un plato (JPG, PNG o WEBP, máx. 2 MB)	Formulario de plato (administrador)

5.2 Requisitos no funcionales

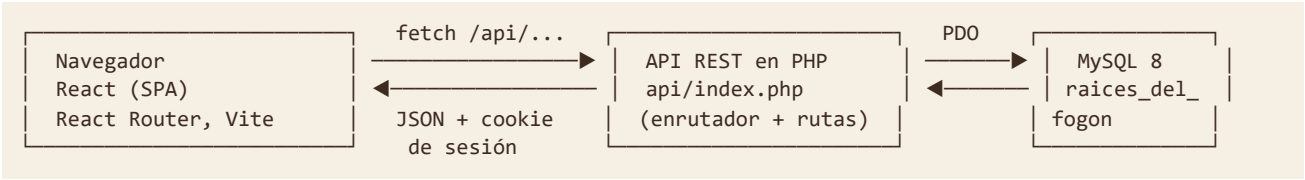
Código	Requisito
RNF01	Diseño adaptable a celulares, tabletas y computadores.
RNF02	Contraseñas almacenadas con hash (bcrypt), nunca en texto plano.
RNF03	Validación de datos en el navegador **y** en el servidor.
RNF04	Protección contra inyección SQL mediante sentencias preparadas.
RNF05	Integridad referencial mediante llaves foráneas en MySQL.
RNF06	Mensajes de error comprensibles, sin exponer detalles internos.
RNF07	Credenciales fuera del código (archivo `.env`).
RNF08	Interfaz en español, con etiquetas en todos los campos.

6. Tecnologías utilizadas

Componente	Tecnología	Versión probada
Interfaz	React + React Router	19.2 / 7.18
Empaquetador	Vite	8.3
Lenguaje del frontend	JavaScript (JSX), HTML5, CSS3	—
API	PHP con PDO	8.3.36
Base de datos	MySQL (motor InnoDB, `utf8mb4_spanish_ci`)	8.4.11
Calidad de código	ESLint	9.39
Pruebas de la API	`node:test` (incluido en Node.js)	Node 25.5

No se agregaron dependencias nuevas de npm: `react-router-dom` ya estaba instalado. La API PHP no usa librerías externas. Las dependencias existentes se actualizaron con `npm audit fix` (sin cambios incompatibles) para corregir vulnerabilidades conocidas de React Router y Vite.

7. Arquitectura



- **Desarrollo:** Vite sirve React en `http://localhost:5173` y reenvía `/api` y `/uploads` (fotos) a la API PHP en `http://127.0.0.1:3001` (`vite.config.js`). Para el navegador todo es el mismo origen, por eso la cookie de sesión funciona sin configurar CORS.
- **Producción:** `npm start` compila React en `dist/` y el mismo `api/index.php` entrega la API y los archivos de `dist/`. Con Apache (Laragon/XAMPP), el archivo `.htaccess` de la raíz envía todas las peticiones a `api/index.php`.

7.1 Estructura de carpetas

```
menu_restaurante/
├── api/                          API REST en PHP
│   ├── index.php                 Punto de entrada: enrutador, cabeceras de seguridad, fotos y frontend comp
│   ├── lib/
│   │   ├── archivos.php          Subida segura de fotos y entrega de uploads/platos/
│   │   ├── config.php            Lectura del archivo .env
│   │   ├── db.php                Conexión PDO, consultas preparadas, transacciones
│   │   ├── respuesta.php         Respuestas JSON y manejo centralizado de errores
│   │   ├── seguridad.php         Sesiones, contraseñas y permisos
│   │   └── validaciones.php      Validaciones del servidor
│   └── rutas/                    auth, categorias, platos, pedidos, reportes
├── database/
│   ├── schema.sql                DDL: base de datos, tablas, llaves, índices, restricciones
│   ├── seed.sql                  DML: categorías y platos iniciales
│   ├── migracion_001_fotos.sql Agrega la foto subida a bases de datos ya existentes
│   ├── instalar.php              Instalador (npm run db:setup)
│   └── consultas.sql             Consultas comentadas para la sustentación
├── uploads/platos/               Fotos subidas (se crea sola; no se sube a git)
├── src/                          Frontend React
│   ├── main.jsx                  Punto de entrada (BrowserRouter + SesionProvider)
│   ├── App.jsx                   Rutas y estado de la orden
│   ├── Tarjeta.jsx, Filtro.jsx, Carrito.jsx   Componentes del menú (originales, adaptados)
│   ├── empresa.js                Textos de identidad corporativa
│   ├── components/               Navbar, Campo, Aviso, RutaProtegida, EstadoPedido
│   ├── context/                  Sesión del usuario (contexto de React)
│   ├── pages/                    Menu, Nosotros, Registro, Login, MisPedidos, PedidoDetalle
│   │   └── admin/                AdminPlatos, PlatoFormulario, PlatoDetalle, AdminCategorias, AdminPedidos,
│   ├── services/api.js           Única capa de comunicación con la API
│   └── utils/                    validaciones.js, formato.js
├── tests/api.test.js             Pruebas automáticas de la API
├── docs/                         Documentación, manual y capturas
├── .env.example                  Plantilla de configuración (sin secretos)
└── .htaccess                     Configuración para Apache
```

7.2 Rutas de la aplicación React

Ruta	Página	Acceso
`/`	Menú y orden	Público
`/nosotros`	Identidad corporativa	Público
`/registro`, `/login`	Registro e inicio de sesión	Público
`/mis-pedidos`, `/mis-pedidos/:id`	Historial y detalle	Cliente
`/admin/platos`	Listado, búsqueda, filtros y eliminación	Administrador
`/admin/platos/nuevo`, `/admin/platos/:id/editar`	Formulario de plato	Administrador
`/admin/platos/:id`	Consulta individual del plato	Administrador
`/admin/categorias`	CRUD de categorías	Administrador
`/admin/pedidos`, `/admin/pedidos/:id`	Gestión de pedidos	Administrador
`/admin/reportes`	Reportes	Administrador

`RutaProtegida` redirige a `/login` si no hay sesión. Es solo una ayuda de navegación: **la protección real la aplica la API**, que revisa la sesión y el rol en cada petición.

8. Comunicación con el backend (API REST)

Todas las respuestas son JSON. Los errores tienen la forma `{ "error": "mensaje", "campos": { "campo": "mensaje" } }`.

Método	Ruta	Acceso	Descripción
POST	`/api/auth/registro`	Público	Registra un cliente e inicia su sesión
POST	`/api/auth/login`	Público	Inicia sesión
POST	`/api/auth/logout`	Público	Cierra sesión
GET	`/api/auth/sesion`	Público	Usuario de la sesión actual (o `null`)
GET	`/api/categorias`	Público	Categorías con conteo de platos
GET	`/api/categorias/{id}`	Público	Una categoría
POST / PUT / DELETE	`/api/categorias[/{id}]`	Admin	Crear, editar, eliminar
GET	`/api/platos?buscar=&categoria_id=&disponible=&precio_min=&precio_max=&orden=&dir=`	Público*	Listado con filtros combinados y orden
GET	`/api/platos/{id}`	Público*	Un plato con sus estadísticas de venta
POST / PUT / DELETE	`/api/platos[/{id}]`	Admin	Crear, editar, eliminar (al eliminar un plato también se borra su foto)
POST	`/api/platos/{id}/imagen`	Admin	Sube o reemplaza la foto (`multipart/form-data`, campo `imagen`)
DELETE	`/api/platos/{id}/imagen`	Admin	Quita la foto subida y borra el archivo
GET	`/uploads/platos/{nombre}`	Público	Entrega una foto subida
POST	`/api/pedidos`	Cliente	Crea un pedido (transacción)
GET	`/api/pedidos?estado=&desde=&hasta=&buscar=`	Cliente / Admin	El cliente ve los suyos; el admin, todos
GET	`/api/pedidos/{id}`	Dueño / Admin	Pedido con su detalle
PATCH	`/api/pedidos/{id}/estado`	Admin	Cambia el estado

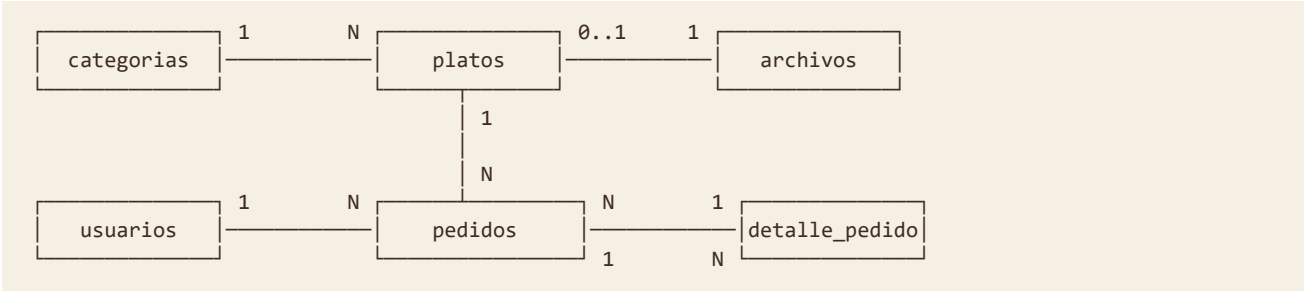
Método	Ruta	Acceso	Descripción
PATCH	`/api/pedidos/{id}/cancelar`	Dueño	Cancela si está pendiente
GET	`/api/reportes/resumen`	Admin	Totales, promedios y platos más vendidos

* Los visitantes y clientes solo ven platos disponibles; el administrador los ve todos.

Códigos de respuesta: 200/201 éxito, 400 datos inválidos, 401 sin sesión, 403 sin permiso, 404 no existe, 409 conflicto (correo repetido, registro con relaciones), 413 foto demasiado grande, 415 formato de foto no permitido, 503 base de datos no disponible, 500 error inesperado.

9. Modelo de datos

9.1 Modelo entidad-relación



Lectura de las relaciones:

- Una **categoría** agrupa muchos **platos**; cada plato pertenece a una sola categoría.
- Un **usuario** puede hacer muchos **pedidos**; cada pedido pertenece a un solo usuario.
- Un **pedido** tiene muchas líneas de **detalle_pedido**; cada línea pertenece a un solo pedido.
- Un **plato** puede aparecer en muchas líneas de **detalle_pedido**.
- `detalle_pedido` es la tabla intermedia que resuelve la relación **muchos a muchos** entre `pedidos` y `platos`.
- Un **plato** puede tener como máximo una foto subida (**archivos**), y cada foto pertenece a un solo plato (llave única en `platos.archivo_id`). La foto es opcional.

9.2 Llaves foráneas y reglas de integridad

Llave	Tabla → referencia	Al actualizar	Al eliminar	Motivo
`fk_platos_categoria`	platos.categoria_id → categorias.id	CASCADE	RESTRICT	No se borra una categoría con platos
`fk_pedidos_usuario`	pedidos.usuario_id → usuarios.id	CASCADE	RESTRICT	Se conserva el historial de pedidos
`fk_detalle_pedido`	detalle_pedido.pedido_id → pedidos.id	CASCADE	CASCADE	El detalle no existe sin su pedido
`fk_detalle_plato`	detalle_pedido.plato_id → platos.id	CASCADE	RESTRICT	No se borra un plato que ya se vendió
`fk_platos_archivo`	platos.archivo_id → archivos.id	CASCADE	SET NULL	Si se borra la foto, el plato queda sin foto subida

9.3 Diccionario de datos

categorias

Campo	Tipo	Nulo	Clave / restricción	Descripción
id	INT UNSIGNED	No	PK, AUTO_INCREMENT	Identificador
nombre	VARCHAR(50)	No	UNIQUE	Nombre de la categoría
descripcion	VARCHAR(255)	Sí		Descripción opcional
creado_en	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP	Fecha de creación

platos

Campo	Tipo	Nulo	Clave / restricción	Descripción
id	INT UNSIGNED	No	PK, AUTO_INCREMENT	Identificador
categoria_id	INT UNSIGNED	No	FK → categorias.id, índice	Categoría del plato
nombre	VARCHAR(100)	No	UNIQUE	Nombre del plato
descripcion	VARCHAR(500)	No		Descripción
precio	DECIMAL(10,2)	No	CHECK precio > 0	Precio en pesos colombianos
imagen_url	VARCHAR(500)	Sí		URL externa de la foto (opcional)
archivo_id	INT UNSIGNED	Sí	FK → archivos.id, UNIQUE	Foto subida al servidor; si existe, se muestra en lugar de la URL
disponible	TINYINT(1)	No	DEFAULT 1, índice	1 = se muestra en el menú
creado_en	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP	Fecha de creación
actualizado_en	TIMESTAMP	No	ON UPDATE CURRENT_TIMESTAMP	Última modificación

usuarios

Campo	Tipo	Nulo	Clave / restricción	Descripción
id	INT UNSIGNED	No	PK, AUTO_INCREMENT	Identificador
nombre	VARCHAR(100)	No		Nombre completo
correo	VARCHAR(150)	No	UNIQUE	Correo (usuario de acceso)
telefono	VARCHAR(20)	No		Teléfono de contacto
password_hash	VARCHAR(255)	No		Hash bcrypt de la contraseña
rol	ENUM('cliente','admin')	No	DEFAULT 'cliente'	Rol en el sistema
creado_en	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP	Fecha de registro

pedidos

Campo	Tipo	Nulo	Clave / restricción	Descripción
id	INT UNSIGNED	No	PK, AUTO_INCREMENT	Número del pedido
usuario_id	INT UNSIGNED	No	FK → usuarios.id, índice	Cliente que lo hizo
estado	ENUM('pendiente','en_preparacion','entregado','cancelado')	No	DEFAULT 'pendiente', índice	Estado actual
total	DECIMAL(10,2)	No	CHECK total >= 0	Valor total
observaciones	VARCHAR(255)	Sí		Indicaciones del cliente
creado_en	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP, índice	Fecha y hora

intentos_login (seguridad; sin llave foránea a propósito, porque también registra correos que no existen)

Campo	Tipo	Nulo	Clave / restricción	Descripción
id	INT UNSIGNED	No	PK, AUTO_INCREMENT	Identificador
correo	VARCHAR(150)	No	Índice (correo, creado_en)	Correo con el que se intentó entrar
ip	VARCHAR(45)	No	Índice (ip, creado_en)	Dirección IP de origen
creado_en	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP	Fecha del intento fallido (se borran a las 24 h y al entrar correctamente)

detalle_pedido

Campo	Tipo	Nulo	Clave / restricción	Descripción
id	INT UNSIGNED	No	PK, AUTO_INCREMENT	Identificador
pedido_id	INT UNSIGNED	No	FK → pedidos.id; UNIQUE (pedido_id, plato_id)	Pedido al que pertenece
plato_id	INT UNSIGNED	No	FK → platos.id, índice	Plato pedido
cantidad	SMALLINT UNSIGNED	No	CHECK entre 1 y 50	Unidades
precio_unitario	DECIMAL(10,2)	No	CHECK > 0	Precio del plato **en el momento de la compra**
subtotal	DECIMAL(12,2)	—	Columna generada: cantidad × precio_unitario	Subtotal de la línea

archivos

Campo	Tipo	Nulo	Clave / restricción	Descripción
id	INT UNSIGNED	No	PK, AUTO_INCREMENT	Identificador
nombre_original	VARCHAR(255)	No		Nombre que tenía el archivo en el equipo del usuario (sin rutas)
nombre_sistema	VARCHAR(100)	No	UNIQUE	Nombre aleatorio con el que se guardó (32 caracteres + extensión)
ruta	VARCHAR(255)	No		Ruta relativa, por ejemplo `uploads/platos/3d89….png`
tipo_mime	VARCHAR(50)	No		Tipo real detectado: `image/jpeg`, `image/png` o `image/webp`
tamano_bytes	INT UNSIGNED	No	CHECK > 0	Tamaño del archivo
fecha_subida	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP	Fecha y hora de la subida

10. Formularios y validaciones

Cada formulario valida en React antes de enviar (archivo `src/utils/validaciones.js`) y la API vuelve a validar con las mismas reglas (`api/lib/validaciones.php`). Si la API rechaza un dato, el mensaje aparece debajo del campo correspondiente.

Formulario	Campo	Reglas
Registro	Nombre	Obligatorio, 3–100 caracteres, solo letras y espacios
	Correo	Obligatorio, formato válido, máx. 150, único
	Teléfono	Obligatorio, 7–15 dígitos
	Contraseña	Obligatoria, 8–72 caracteres, letras y números
	Confirmar contraseña	Debe coincidir
Inicio de sesión	Correo, contraseña	Obligatorios; formato de correo
Plato	Nombre	Obligatorio, 3–100, único
	Descripción	Obligatoria, 10–500
	Precio	Obligatorio, número positivo con máx. 2 decimales, entre $500 y $1.000.000
	Categoría	Obligatoria y existente (llave foránea)
	Foto (archivo)	Opcional; JPG, PNG o WEBP; máx. 2 MB y 6000 px por lado; el servidor revisa el contenido real
	URL de imagen	Opcional, debe empezar por `http://` o `https://`, máx. 500
Categoría	Nombre	Obligatorio, 3–50, único
	Descripción	Opcional, máx. 255
Orden	Platos	Al menos 1, máx. 30 distintos, cantidad 1–50, platos disponibles
	Observaciones	Opcional, máx. 255

Todos los formularios muestran un estado de carga ("Guardando…", "Ingresando…"), desactivan el botón mientras envían (evita envíos duplicados) y conservan lo escrito si ocurre un error.

11. Operaciones CRUD

Módulo principal: Platos (`/admin/platos`), con CRUD completo:

Operación	Cómo se hace	Endpoint
Create	Botón **+ Nuevo plato** → formulario → **Crear plato**	`POST /api/platos`
Read	Listado con búsqueda, filtros y orden; botón **Ver** para la consulta individual	`GET /api/platos`, `GET /api/platos/{id}`
Update	Botón **Editar** → formulario con los datos cargados → **Guardar cambios**	`PUT /api/platos/{id}`
Delete	Botón **Eliminar** → confirmación → eliminación	`DELETE /api/platos/{id}`

Casos especiales: un plato que no existe muestra "El plato no existe."; un plato que ya aparece en pedidos no se puede eliminar (se sugiere marcarlo como "no disponible"); un nombre repetido devuelve un aviso de conflicto.

También tienen CRUD las **categorías** (`/admin/categorias`), y los **pedidos** tienen creación (cliente), consulta (cliente y administrador) y actualización de estado y cancelación.

12. Consultas a la base de datos

El archivo `database/consultas.sql` reúne las consultas comentadas para la sustentación. Todas se ejecutaron sobre MySQL 8.4 sin errores. Las mismas consultas están integradas en la aplicación:

Tipo	Ejemplo	Dónde se usa
General	Listar platos; consultar plato por id; ordenar por precio	Menú, Gestión de platos, Ver plato
Específica	`LIKE` por texto; filtro por categoría; rango de precios; estado; fechas	Buscador, filtros de platos y pedidos
Multitabla	`platos INNER JOIN categorias LEFT JOIN archivos`	Menú y listado de platos (con su foto)
Multitabla	`categorias LEFT JOIN platos` + `COUNT`	Gestión de categorías
Multitabla	`pedidos JOIN usuarios LEFT JOIN detalle_pedido`	Mis pedidos, Gestión de pedidos
Multitabla (5 tablas)	`pedidos – usuarios – detalle_pedido – platos – categorias`	Detalle del pedido
Resumen	`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, `GROUP BY`, `LIMIT`	Página Reportes

El archivo también incluye ejemplos de **INSERT, UPDATE y DELETE** dentro de una transacción con `ROLLBACK` (no modifican los datos) y sentencias que **deben fallar** para demostrar las llaves foráneas y las restricciones CHECK.

13. Medidas de seguridad

Riesgo	Medida aplicada
Inyección SQL	PDO con sentencias preparadas reales (`ATTR_EMULATE_PREPARES = false`); los nombres de columna para ordenar salen de una lista blanca
Contraseñas expuestas	`password_hash()` (bcrypt) y `password_verify()`; el hash nunca se envía al navegador
Robo de sesión por JavaScript (XSS)	Sesión en cookie `httpOnly`; React escapa todo el texto que muestra; las URL de imágenes solo aceptan `http(s)://`
Peticiones desde otros sitios (CSRF)	Cookie de sesión con `SameSite=Strict`
Fijación de sesión	`session_regenerate_id(true)` al iniciar sesión; `session.use_strict_mode`
Acceso indebido	La API verifica la sesión y el rol en cada operación privada; un cliente no puede ver pedidos ajenos (responde 404)
Escalada de privilegios	El registro siempre crea el rol `cliente`
Manipulación de precios	El total del pedido lo calcula el servidor con los precios de la base de datos
Enumeración de usuarios	Un mismo mensaje para "correo no existe" y "contraseña incorrecta"; si el correo no existe se verifica igual un hash ficticio, para que la respuesta tarde lo mismo
Fuerza bruta en el inicio de sesión	Tabla `intentos_login`: tras 5 intentos fallidos con un correo (o 50 desde una misma IP) en 15 minutos, la API responde 429 y no deja entrar ni con la contraseña correcta hasta que pase el tiempo
Datos sensibles en los registros	El log guarda solo el tipo de error, el mensaje, el archivo y la línea (no la traza con argumentos); `zend.exception_ignore_args = 1`
Fuga de información técnica	Errores internos solo en el log; al cliente le llega un mensaje genérico; `display_errors = 0`
Secretos en el código	Credenciales en `.env` (ignorado por git); `.env.example` sin valores
Archivos internos expuestos	`api/index.php` solo entrega archivos de `dist/` y fotos cuyo nombre cumple el patrón `uploads/platos/<32 hex>.(jpg|png|webp)`; `.htaccess` bloquea archivos ocultos
Subida de archivos maliciosos	Tipo real verificado con `getimagesize()` (no se confía en la extensión ni en el tipo que envía el navegador); extensión asignada por el servidor según el tipo real; nombre aleatorio con `random_bytes`; límite de 2 MB y 6000 px; solo el administrador puede subir; las fotos nunca se ejecutan, solo se leen y se entregan con su tipo de imagen
Archivos huérfanos	Si falla el registro en MySQL, se borra el archivo recién subido; al reemplazar o quitar una foto, o al eliminar el plato, se borra el archivo anterior
Cabeceras	`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: same-origin`
Integridad de datos	Llaves foráneas, UNIQUE, CHECK y transacciones al crear pedidos

14. Instalación y configuración

1. Instale Node.js 20+, PHP 8.1+ (con `pdo_mysql` y `mbstring`) y MySQL 8. Con Laragon o XAMPP, encienda MySQL y agregue la carpeta de PHP a la variable `PATH` para poder usar el comando `php`.
2. En la carpeta del proyecto ejecute `npm install`.
3. Copie `.env.example` como `.env` y complete:

Variable	Descripción
`DB_HOST`, `DB_PORT`	Servidor MySQL (por defecto `127.0.0.1` y `3306`)
`DB_USER`, `DB_PASSWORD`	Usuario de MySQL (en Laragon, `root` sin contraseña)
`DB_NAME`	`raices_del_fogon`
`ADMIN_NOMBRE`, `ADMIN_CORREO`, `ADMIN_TELEFONO`, `ADMIN_PASSWORD`	Datos del administrador que se crea al instalar

4. Ejecute `npm run db:setup` (o `npm run db:setup:demo` para incluir datos ficticios).
5. Desarrollo: `npm run api` y, en otra terminal, `npm run dev` → http://localhost:5173.
6. Producción: `npm start` → http://127.0.0.1:3001.

Alternativa con phpMyAdmin: importe `database/schema.sql` y luego `database/seed.sql`. En ese caso el administrador no se crea, porque su contraseña debe guardarse como hash; ejecute `npm run db:setup` para crearlo.

Actualizar una base de datos anterior: `npm run db:setup` detecta si falta la columna `platos.archivo_id` y aplica `database/migracion_001_fotos.sql` sin borrar datos. Con phpMyAdmin, ejecute ese archivo una sola vez.

Fotos subidas: se guardan en `uploads/platos/` (la carpeta se crea sola). No están en git, así que inclúyalas en las copias de seguridad junto con la base de datos. El límite de 2 MB coincide con el valor por defecto de PHP (`upload_max_filesize = 2M`, `post_max_size = 8M`); para fotos más grandes, suba esos valores en `php.ini` (`post_max_size` siempre mayor), reinicie el servidor y ajuste `MAX_BYTES_IMAGEN` en `api/lib/archivos.php` y en `src/utils/validaciones.js`.

15. Procedimiento de pruebas

Prueba	Comando o procedimiento	Resultado obtenido (22-10-2026)
Revisión de código	`npm run lint`	Sin errores
Compilación	`npm run build`	Correcta
Sintaxis PHP	`php -l` en todos los archivos	Sin errores
Instalación de la BD	`npm run db:setup:demo` ejecutado dos veces	Correcta; la segunda vez no duplicó datos
Instalación limpia	`schema.sql` + `seed.sql` en una base vacía	7 tablas, 5 llaves foráneas y 9 platos
Migración de fotos	`npm run db:setup` sobre la base existente, dos veces	La migración se aplicó una sola vez; platos y pedidos intactos
Pruebas automáticas de la API	`npm run test:api`	**7 de 7 pruebas aprobadas** (consultas públicas, registro e inicio de sesión, bloqueo por fuerza bruta, permisos, CRUD de platos y categorías, pedidos y reportes, fotos de platos)
Consultas SQL	Ejecución completa de `database/consultas.sql`	Todas sin errores; las 4 sentencias de demostración de integridad fallaron como se esperaba (errores 1452, 1451, 1451 y 3819)
Prueba manual en navegador	Registro inválido y válido, correo repetido, inicio de sesión, orden y pedido, detalle, crear/editar/eliminar plato, eliminar plato vendido (bloqueado), cambiar estado, reportes, cerrar sesión, ruta protegida sin sesión	Correcto; los cambios se verificaron directamente en MySQL
XAMPP 8.2.12 (Apache + MariaDB 10.4)	Instalación de la BD, `consultas.sql`, pruebas automáticas con `API_URL=http://localhost:8080/api` y revisión en el navegador	Correcto: 7 tablas, 5 llaves, 7 de 7 pruebas; `.env` y `.htaccess` responden 403. En MariaDB el error de CHECK es el 4025 (en MySQL, 3819)
Subida de fotos en navegador	Archivo que no es imagen (rechazado en el navegador), crear plato con foto (vista previa, guardado, foto visible en el menú), quitar foto subida (vuelve la URL externa)	Correcto; sin archivos ni registros huérfanos al terminar
Dependencias	`npm audit`	0 vulnerabilidades (después de `npm audit fix`)
Vista móvil	Navegador con 375 × 812 px: menú, botón de navegación, tablas	Correcto
Consola del navegador	Revisión de errores	Solo los dos 409 esperados (correo repetido y plato con pedidos)

Las pruebas automáticas (`tests/api.test.js`) cubren, entre otros casos: validaciones con datos inválidos, correo duplicado, contraseña errada, acceso sin sesión (401) y sin permiso (403), intento de inyección SQL en la búsqueda, llave foránea con categoría inexistente, plato oculto para el público, eliminación protegida, total del pedido calculado por el servidor y pedidos ajenos. Para las fotos: subida sin sesión (401) y como cliente (403),

formulario sin archivo (400), código PHP disfrazado de `.png` (415), archivo de más de 2 MB (413), nombre con rutas (`../../mi foto.png` se guarda como `mi foto.png`), reemplazo y eliminación con borrado del archivo anterior, y petición de rutas fuera de `uploads/platos/` (404).

Durante las pruebas se corrigieron cuatro errores: campos de formulario que se borraban al escribir muy rápido, un mensaje con concordancia de género incorrecta ("La descripción es obligatorio"), una redirección equivocada al cerrar sesión desde una página de administración y un formulario de foto vacío que se informaba como "archivo demasiado grande".

16. Limitaciones conocidas

- La misión, visión, objetivos y valores son una **propuesta** que el equipo debe validar. Los **datos de contacto** del restaurante están pendientes.
- No hay recuperación de contraseña.
- Las fotos subidas no se redimensionan ni se comprimen: se guardan tal como llegan (máximo 2 MB).
- En el navegador integrado de pruebas la foto se cargó generando la imagen desde la página, porque esa herramienta no puede abrir el selector de archivos del sistema operativo; el envío al servidor es el mismo que al elegirla manualmente.
- No hay pagos en línea ni domicilios; el pedido queda registrado para que el restaurante lo gestione.
- Solo existe un administrador inicial. Para crear otro, cambie el rol de un usuario directamente en MySQL.
- Se probó en dos entornos: MySQL 8.4 + PHP 8.3 con el servidor integrado de PHP, y XAMPP 8.2.12 (Apache 2.4 + MariaDB 10.4.32 + PHP 8.2.12). No se probó en un servidor de hosting público.
- Para usar la cookie de sesión con `Secure` se requiere HTTPS en producción.

Referencias

Meta Platforms. (s.f.). *React*. https://react.dev/

OWASP Foundation. (s.f.). *OWASP Top Ten*. https://owasp.org/www-project-top-ten/

Oracle Corporation. (s.f.). *MySQL 8.4 reference manual*. https://dev.mysql.com/doc/refman/8.4/en/

The PHP Group. (s.f.). *Manual de PHP*. https://www.php.net/manual/es/

Vite. (s.f.). *Vite: herramientas de frontend de nueva generación*. https://vite.dev/